

Deep Learning - Computer Assignment 1

Technical Analysis of the Python/Colab Reconstruction

Publication-quality technical report based strictly on the supplied source code

Contents

Abstract	1
1 Introduction	1
2 Project Overview	1
2.1 Source inventory	2
3 Technical Foundations	2
3.1 Linear multiclass classification	2
3.2 The multiclass perceptron update	3
3.3 Nonlinear feed-forward classification	3
3.4 Sigmoid and binary cross-entropy	3
3.5 Backpropagation	4
4 Data and Input Processing	4
4.1 Dataset discovery and fallback behavior	4
4.2 The Q1 label-consistency issue	4
4.3 Train/test construction	5
5 Detailed Methodology	5
5.1 Task 1: single-layer multiclass perceptron	5
5.2 Task 2: two-layer nonlinear network	5
5.3 Evaluation and serialization	6
6 System Architecture and Code Organization	6
7 Implementation Details and Design Decisions	7
7.1 Reproducibility controls	7
7.2 Validation philosophy	7
7.3 Numerical safeguards	7
7.4 Toolbox restriction and algorithm ownership	7
7.5 Auxiliary plotting path	8
8 Execution Workflow	8
9 Design Decisions and Rationale	8
10 Computational Considerations	9
10.1 Potential numerical and algorithmic constraints	9
11 Reproducibility and Reliability	9
12 Limitations and Technical Considerations	10
13 Overall Technical Assessment	10
14 Conclusion	11
A Complete Source-Coverage Traceability	12
B Implementation Parameters Explicitly Present in the Source	13

Abstract

The supplied source file is a Google Colab-oriented Python reconstruction of a two-part deep-learning assignment. Its stated objective is to solve a three-class geometric classification problem with a single-layer multiclass perceptron and to determine whether a moon-shaped binary dataset requires a nonlinear two-layer network. The implementation avoids ready-made machine-learning toolboxes and instead implements the learning rules, activation functions, loss, gradient calculations, predictions, and confusion matrices directly with NumPy. It also provides reproducibility through a fixed random seed and applies training-only standardization to the second task. The source states that the code was reconstructed from a supplied MATLAB assignment and that several defects in the original implementation were identified and corrected.

This report analyzes the complete 484-line source at the structural, algorithmic, mathematical, and software-engineering levels. The first task is formulated as a multiclass linear scoring model with winner-takes-all prediction and the standard perceptron correction for the true and predicted classes. The second task uses a two-layer feed-forward network with a hyperbolic-tangent hidden layer and sigmoid output, trained by full-batch gradient descent on binary cross-entropy. The report distinguishes explicit implementation facts from reasonable inferences and records important constraints, including the absence of the actual dataset files from the supplied source package, unused imports, the deterministic but non-shuffled 70/30 split, and the fact that reported execution results are generated at runtime rather than embedded in the source. No experimental results, figures, or result visualizations are reproduced here.

1 Introduction

The project is best understood as a reconstruction and repair of a small neural-classification laboratory rather than as a generic machine-learning script. The source begins by identifying itself as a Colab-generated notebook reconstruction and states two assignment tasks: a three-class perceptron on the `testX` dataset and a two-layer perceptron-style solution for a moon-shaped dataset. The assignment also imposes a methodological restriction: the learning algorithms are to be implemented without ready-made machine-learning or neural-network toolboxes.

The source has two related goals. The first is computational: build models that can be trained and evaluated on fixed datasets. The second is methodological: preserve the educational structure of the original assignment while correcting implementation choices that would otherwise undermine the intended algorithms. This second goal is explicit in the audit section, which identifies problems in the original multiclass-label construction, learning rule, evaluation, nonlinear model, prediction, and reproducibility (source lines 21–32).

The purpose of this report is not to restate the notebook as prose. Instead, it reconstructs the computational logic so that a technically informed reader can understand the assumptions, data flow, mathematical operations, and software decisions without first opening the source. The analysis covers environment setup, data ingestion, target reconstruction, preprocessing, both learning algorithms, evaluation utilities, result serialization, reproducibility metadata, and the final archive-generation path.

2 Project Overview

At a high level, the program follows this pipeline:

1. Establish a Python/Colab execution environment and create project directories.
2. Locate four CSV datasets, with a Google Colab upload fallback when they are absent.
3. Validate shapes, finiteness, and label domains.
4. Reconstruct the three-class target for the first dataset from coordinate-based rules.
5. Perform the required 70/30 split without shuffling.
6. Train a multiclass perceptron on the first dataset.
7. Evaluate predictions and construct a confusion matrix for the first task.
8. Standardize the second dataset using only training-set statistics.
9. Train a two-layer nonlinear network from scratch with manual backpropagation.
10. Evaluate the second network using deterministic thresholded predictions and a binary confusion matrix.
11. Save model parameters, training history, confusion matrices, a summary table, a reproducibility manifest, and a ZIP archive.

This ordering is conceptual rather than a literal reproduction of notebook-cell execution. Plotting and serialization are auxiliary outputs around the two core learning procedures rather than independent methodological stages, although the source executes them as top-level cells after model training.

2.1 Source inventory

The supplied file contains 484 lines. It defines eight functions and no classes. The main dependencies are NumPy, pandas, Matplotlib, SciPy’s MATLAB reader, and standard-library modules for file handling, JSON serialization, dynamic package installation, and ZIP creation. The source installs NumPy, pandas, Matplotlib, SciPy, and `nbformat` when they are absent; `nbformat` is an environment dependency of the notebook-conversion workflow but is not used in the shown source. Likewise, `loadmat`, `os`, and `math` are imported but not used by the current execution path.

Component	Source-level role
Environment setup	Optional package installation, imports, seed initialization, working directories
Dataset loading	Reads four CSV files and provides a Colab upload fallback
Validation	Checks exact shapes, finite inputs, and binary label domains
Target reconstruction	Maps the first dataset into three geometric classes
Q1 learner	Manual multiclass perceptron using NumPy matrix/vector operations
Evaluation utility	Manual confusion-matrix construction
Q2 learner	Manual two-layer tanh/sigmoid network with full-batch backpropagation
Persistence	CSV, NPZ, JSON, and ZIP serialization of generated artifacts

3 Technical Foundations

3.1 Linear multiclass classification

For the first task, each class is represented by a linear score. Let $x \in \mathbb{R}^d$ be an input vector, let $W \in \mathbb{R}^{K \times d}$ collect the class weight vectors, and let $b \in \mathbb{R}^K$ collect the class biases. The score vector is

$$s(x) = Wx + b. \quad (1)$$

With three classes, $K = 3$. Prediction uses a winner-takes-all rule:

$$\hat{y} = 1 + \arg \max_{k \in \{0,1,2\}} s_k(x), \quad (2)$$

where the source stores target labels as 1, 2, 3 but uses zero-based class indices internally during the update. The associated class boundaries are linear because equality of two class scores gives an affine hyperplane. For two classes represented by rows $w_i^\top$ and $w_j^\top$, the pairwise boundary is

$$(w_i - w_j)^\top x + (b_i - b_j) = 0. \quad (3)$$

The implementation does not explicitly compute pairwise boundaries during prediction; it computes all class scores and selects the largest.

3.2 The multiclass perceptron update

When a sample is misclassified, the source identifies the true class t and predicted class p , then applies

$$w_t \leftarrow w_t + x, \quad b_t \leftarrow b_t + 1, \quad (4)$$

$$w_p \leftarrow w_p - x, \quad b_p \leftarrow b_p - 1. \quad (5)$$

This update increases the true-class score by $\|x\|^2 + 1$ and decreases the predicted-class score by the same amount, so the true-versus-predicted score gap changes by $2(\|x\|^2 + 1)$. The update is applied sample by sample and epoch by epoch. Training stops early if an epoch has zero errors.

3.3 Nonlinear feed-forward classification

The second task uses a two-layer network because the moon-shaped dataset is treated as non-linearly separable in the original two-dimensional input space. The network has an affine hidden layer, a tanh activation, a second affine transformation, and a sigmoid output. With input dimension $d = 2$ and hidden width $h = 12$,

$$z^{(1)} = XW_1 + b_1, \quad (6)$$

$$a^{(1)} = \tanh(z^{(1)}), \quad (7)$$

$$z^{(2)} = a^{(1)}W_2 + b_2, \quad (8)$$

$$a^{(2)} = \sigma(z^{(2)}), \quad (9)$$

The final output is interpreted as a value in $[0, 1]$ and thresholded at 0.5 for binary prediction.

3.4 Sigmoid and binary cross-entropy

The sigmoid is

$$\sigma(z) = \frac{1}{1 + e^{-z}}. \quad (10)$$

The source clips the sigmoid input to $[-50, 50]$. This is a numerical safeguard that limits extreme exponentials while leaving ordinary values unchanged. The binary cross-entropy implemented for N samples is

$$\mathcal{L}(y, \hat{y}) = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]. \quad (11)$$

The predicted probabilities are clipped to $[10^{-12}, 1 - 10^{-12}]$ before the logarithms are evaluated. This prevents numerical problems at exactly zero or one.

3.5 Backpropagation

For a sigmoid output with binary cross-entropy, the source uses the standard simplification

$$\delta^{(2)} = a^{(2)} - y, \quad (12)$$

which yields

$$\frac{\partial \mathcal{L}}{\partial W_2} = \frac{(a^{(1)})^\top \delta^{(2)}}{N}, \quad \frac{\partial \mathcal{L}}{\partial b_2} = \text{mean}(\delta^{(2)}), \quad (13)$$

$$\delta^{(1)} = \left(\delta^{(2)} W_2^\top \right) \odot \left(1 - (a^{(1)})^2 \right), \quad (14)$$

$$\frac{\partial \mathcal{L}}{\partial W_1} = \frac{X^\top \delta^{(1)}}{N}, \quad \frac{\partial \mathcal{L}}{\partial b_1} = \text{mean}(\delta^{(1)}). \quad (15)$$

The derivative $1 - a^2$ is the derivative of $\tanh(z)$ written in terms of its output. The parameters are updated with full-batch gradient descent:

$$\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}, \quad (16)$$

with learning rate $\eta = 0.03$.

4 Data and Input Processing

4.1 Dataset discovery and fallback behavior

The source creates a `datasets` directory below the current working directory and searches for four specific CSV files: `testdataX.csv`, `testdataY.csv`, `moondataX.csv`, and `moondataY.csv`. When one or more files are missing, the code attempts to import `google.colab.files` and presents an interactive upload workflow. Outside Colab, the same branch raises a `FileNotFoundError` instructing the user to place the files in `./datasets/`. This makes the notebook convenient both for a bundled project repository and for standalone Colab use.

The source reads the feature matrices as floating-point NumPy arrays and the labels as integer arrays. The expected shapes are exactly 100×2 for each feature matrix and length 100 for each label vector. It also requires all feature values to be finite and verifies that the supplied labels are binary. These assertions provide inexpensive contract checks before training.

4.2 The Q1 label-consistency issue

A central reconstruction issue is the mismatch between the supplied binary `testdataY.csv` and the three-class interpretation required by the assignment. The source documents that the binary labels match the sign of x_2 , while the prepared MATLAB logic defines three geometric regions. The reconstruction uses the documented three-class rule because it matches the stated task. The function `build_three_class_targets` implements

$$y(x) = \begin{cases} 1, & x_2 < 0, \\ 2, & x_2 > 0 \text{ and } x_1 < 0, \\ 3, & x_2 > 0 \text{ and } x_1 > 0. \end{cases} \quad (17)$$

The explicit axis check matters: the function refuses to classify samples with $x_1 = 0$ or $x_2 = 0$ because the stated rules do not define tie handling on an axis. This avoids silently assigning an arbitrary class.

4.3 Train/test construction

Both tasks use the first 70 samples for training and the final 30 for testing. There is no shuffling, so the split is positional and deterministic. For Q1, the original coordinate system is retained. For Q2, the training subset is summarized by a mean vector μ and standard-deviation vector s , and both training and test samples are transformed using

$$x_{\text{std}} = \frac{x - \mu}{s}. \quad (18)$$

The mean μ and standard deviation s are calculated only from the first 70 training samples, so the held-out test samples do not influence the scaling transform.

5 Detailed Methodology

5.1 Task 1: single-layer multiclass perceptron

The function `multiclass_perceptron_train` receives features, labels, the number of classes, an epoch limit, and a seed. It creates a local NumPy generator and initializes the weight matrix and bias vector from independent normal distributions with standard deviation 0.1. The dimensions are explicit: $W \in \mathbb{R}^{3 \times 2}$ and $b \in \mathbb{R}^3$.

The inner loop is intentionally simple. For each sample x_i , the code computes $Wx_i + b$, obtains the predicted class through `argmax`, converts the one-based target $y_i \in \{1, 2, 3\}$ to a zero-based index, and conditionally applies the true/predicted-class update. The variable `errors` counts the number of updates in that epoch and is appended to `history`. Training terminates early when that count reaches zero. The resulting history is not a loss curve; it is an integer-valued record of classification mistakes per epoch.

Because the implementation uses a fixed sample order, the algorithm is deterministic under the chosen seed, but it is not a shuffled online-perceptron procedure. This affects both reproducibility and the exact trajectory of the weights. The source does not claim a random train/test split, and none occurs.

After training, the code computes train and test score matrices with matrix multiplication and bias broadcasting. It converts these scores into one-based class predictions, computes accuracy with a direct NumPy equality mean, and constructs a 3-by-3 confusion matrix with a hand-written routine. The matrix rows represent true classes and the columns represent predicted classes; this orientation should be preserved when interpreting the saved CSV.

5.2 Task 2: two-layer nonlinear network

The second learner is implemented in `train_two_layer_network`. Initialization uses a local random generator with the same seed value but an independent generator state. The hidden weights have shape 2×12 , the hidden bias has 12 entries, the output weights have shape 12×1 , and the output bias is scalar. Hidden and output biases start at zero, while weights are drawn from a normal distribution with standard deviation 0.5.

Each training epoch is a complete pass over all 70 training observations in one vectorized forward and backward computation. There are no mini-batches, stochastic updates, or optimizer state variables such as momentum. The optimization is therefore standard full-batch gradient descent. The implementation uses the derivative of $\tanh$ in the form $1 - a^2$, and the output gradient is the simplified sigmoid-plus-cross-entropy derivative.

The history recorder stores the epoch number, loss, and training accuracy at epoch 1, every 100 epochs, and the final epoch. This reduces storage while retaining a useful coarse training

trace. The prediction helper reuses the same forward equations but performs no gradient computation. It returns both continuous sigmoid outputs and binary labels generated by the 0.5 threshold.

5.3 Evaluation and serialization

The source implements a generic `confusion_matrix` function that accepts explicit class labels. This avoids dependence on a machine-learning package and makes the class ordering explicit. Q1 uses class labels 1, 2, 3; Q2 uses 0, 1. The matrices are converted to pandas DataFrames and written to CSV.

For Q2, the trained parameter arrays and the preprocessing statistics μ and s are written to an NPZ archive. The training history is written to CSV. A final summary table and a JSON manifest record training/test sizes, model settings, achieved accuracies from the executed run, and the names of saved result files. The manifest is therefore a compact provenance record, although its accuracy values are runtime-dependent rather than fixed constants in the source.

The final section creates `results.zip` by iterating over files in the results directory and storing them under a `results/` archive prefix. In Colab, the archive is then passed to `files.download`; outside Colab, it remains on disk. Result packaging is therefore part of the reproducibility workflow.

6 System Architecture and Code Organization

The implementation is intentionally flat: it is a notebook-style procedural program rather than a package with classes and modules. This suits a course assignment because each stage remains visible and easy to execute interactively. The functional decomposition is nevertheless meaningful.

Table 1: Function-level traceability to the source code.

Function	Lines	Responsibility
<code>build_three_class_targets</code>	121–129	Encodes the documented geometric three-class rule and rejects axis samples.
<code>multiclass_perceptron_train</code>	170–192	Initializes and trains the Q1 linear multiclass classifier with the perceptron update.
<code>confusion_matrix</code>	210–216	Builds a discrete confusion matrix without a machine-learning library.
<code>plot_linear_boundary</code>	224–228	Computes and draws an affine boundary for the Q1 visualization path; it is auxiliary to the learning algorithm.
<code>sigmoid</code>	263–265	Computes a numerically guarded sigmoid activation.
<code>binary_cross_entropy</code>	268–270	Computes mean binary cross-entropy with probability clipping.

Function	Lines	Responsibility
<code>train_two_layer_network</code>	273–314	Performs forward propagation, backpropagation, and gradient descent for Q2.
<code>predict_two_layer_network</code>	327–330	Performs inference and 0.5 thresholding for Q2.

The absence of classes is not a defect by itself. The learning procedures are self-contained, and the small number of objects means that a class-based abstraction would mainly change the presentation rather than the capability. For a reusable research package, however, the current stateful dictionaries and top-level path management would be natural candidates for later refactoring into explicit model and configuration objects. This is a software-engineering observation, not a claim that the assignment requires such refactoring.

7 Implementation Details and Design Decisions

7.1 Reproducibility controls

A global seed of 42 is defined and applied with `np.random.seed(SEED)`, while each training function also creates its own `default_rng(seed)`. The dual mechanism is somewhat redundant because the current model initialization uses the local generators, but it can improve consistency if later cells introduce legacy NumPy random calls. The exact seed is exposed as a named parameter rather than hidden inside the training functions.

The source also stores the Q2 training mean and standard deviation in the NPZ model artifact. This improves reproducibility because a trained neural-network parameter set alone is not sufficient to reproduce inference when preprocessing changes the input domain.

7.2 Validation philosophy

The assertions near the data-loading stage serve as executable contracts. They establish the expected dataset dimensions and basic numeric validity before expensive computations begin. The target builder adds a semantic check for axis cases. Together, these checks reduce the chance that malformed data are silently propagated into training.

7.3 Numerical safeguards

The sigmoid input is clipped to $[-50, 50]$, and the cross-entropy probability is clipped to $[10^{-12}, 1 - 10^{-12}]$. The Q2 standard deviation uses 1 as a fallback for a constant feature. These are small but useful safeguards against overflow, invalid logarithms, and division by zero. The source does not implement additional stabilization such as a log-sum-exp formulation because its output layer is binary sigmoid rather than a multiclass softmax.

7.4 Toolbox restriction and algorithm ownership

The source explicitly avoids ready-made classification and neural-network toolboxes. This is reflected in the direct implementation of the update equations, activations, loss, gradients, prediction, accuracy, and confusion matrix. SciPy is retained for compatibility with the data-preparation context, but the active data-loading path uses pandas CSV reading. The key learning mechanics are therefore transparent rather than delegated to a framework.

7.5 Auxiliary plotting path

The code generates several figures and saves them under `results/`: Q1 classification geometry, Q2 loss and accuracy histories, and a learned decision-boundary visualization. These artifacts are deliberately excluded from this report, in accordance with the reporting constraint. Their existence is nevertheless relevant to the source architecture because it explains why Matplotlib is a dependency and why the results directory contains image files.

8 Execution Workflow

A clean execution follows this sequence. The runtime begins by checking dependencies and creating the project directories. The four datasets are then located or uploaded. Shape and value assertions run before any model is constructed. The first feature matrix is converted into the three target classes, and both datasets are partitioned into fixed training and test subsets.

The Q1 model is initialized and trained one sample at a time. After each epoch, the number of corrections is stored, and a zero-error epoch causes early termination. The final parameters are then applied to the training and held-out samples. Accuracy and a confusion matrix are computed, and the Q1 confusion matrix is saved. The plotting cell uses the same learned parameters but does not affect training.

The Q2 preprocessing statistics are computed from the training subset only. The standardized training matrix is passed to the two-layer learner. For each of 5,000 epochs, the network performs a vectorized forward pass, computes the output and hidden gradients, and updates all parameters simultaneously. Training checkpoints are collected at a reduced cadence. After training, the network is run on both training and test inputs, the probabilities are thresholded, and the binary confusion matrix is saved. Model parameters, preprocessing statistics, and history are then serialized.

Finally, the program constructs a runtime summary and reproducibility manifest, archives the result files, and optionally triggers a Colab download. The notebook is therefore both a trainer and a small experiment-packaging script.

9 Design Decisions and Rationale

Several reconstruction decisions are explicit rather than inferred. The most important is the choice to replace the original one-vs-all/averaging logic with a standard multiclass perceptron. A three-class assignment requires three competing class scores, and averaging independently learned binary-unit parameters would destroy the information needed for a coherent winner-takes-all classifier. The reconstructed update directly encodes the competition between the true and predicted classes.

The second major decision is to replace the original threshold-unit construction for the moon dataset with a genuine two-layer network. The source argues that a single linear decision surface cannot express the required curved boundary. A hidden nonlinear transformation addresses that limitation by composing multiple affine projections with a nonlinear activation. The code therefore changes not just implementation details but the representational class of the model.

The choice of training-only standardization for Q2 is also methodologically significant. Because the test set is intended to remain held out, using its statistics during preprocessing would leak information across the evaluation boundary. The source avoids that leakage and stores the training statistics with the model artifact.

Finally, the deterministic fixed split and fixed random seed make the execution reproducible for the supplied data and runtime, but they do not establish statistical generalization across alternative samples or random partitions. The split is therefore treated as an assignment-specific protocol rather than as a statistically representative cross-validation design.

10 Computational Considerations

For Q1, let N be the number of training samples, d the input dimension, K the number of classes, and E the number of completed epochs. Each sample requires a matrix-vector score calculation costing $O(Kd)$, so the training work is approximately

$$O(ENKd). \quad (19)$$

With only 70 training samples and $h=12$, the model is computationally modest. The leading scaling is linear in the sample count and hidden width for each full-batch epoch.

For Q2, let h be the hidden width. A full-batch epoch evaluates the forward and backward matrix products associated with the two layers, giving a leading-order cost of

$$O(N(dh + h)), \quad (20)$$

and total training work of approximately

$$O(EN(dh + h)). \quad (21)$$

With only 70 training samples and $h = 12$, the model is computationally modest. The main scaling characteristic is linear in the sample count and hidden width for each full-batch epoch.

Parameter memory for Q2 is also small:

$$O(dh + h + h + 1) = O(dh + h). \quad (22)$$

The implementation does not use mini-batches, mixed precision, GPU-specific kernels, or optimizer state. These omissions simplify the pedagogical implementation but would matter for larger datasets.

10.1 Potential numerical and algorithmic constraints

The Q1 algorithm uses online perceptron updates but does not shuffle the data. The final parameters can therefore depend on sample order, even though that order is fixed for reproducibility. Q2 uses plain gradient descent with a fixed learning rate. No learning-rate schedule, adaptive optimizer, validation set, early stopping, or regularization term is implemented. These are not necessarily omissions for an assignment of this scale, but they define the behavior of the current models.

11 Reproducibility and Reliability

The source contains several useful reproducibility mechanisms. The random seed is explicit; the train/test split is deterministic; Q2 preprocessing is fit on training data only; model parameters and preprocessing statistics are saved; the training history is serialized; and a JSON manifest records key configuration and runtime-derived metrics. The result archive provides a single artifact containing the generated files.

There are also reproducibility boundaries. The source alone does not include the four CSV datasets, so executing it from scratch requires those files or an interactive upload. The source

reports library versions at runtime but does not pin package versions in a requirements file or environment lockfile. The code also relies on the behavior of the Colab upload interface when data are missing. Consequently, the project is reproducible at the algorithmic level and reasonably reproducible at the notebook level, but it is not a fully environment-locked research package.

A separate point concerns the runtime summary. The source calculates training and test accuracies after execution and stores them in CSV and JSON outputs. Because those numerical values are not present as fixed constants in the supplied file, they cannot be responsibly reproduced in this report without executing the code against the actual datasets. The report therefore does not invent or restate numerical performance.

12 Limitations and Technical Considerations

The first limitation is data availability: the supplied source expects four external CSV files. The source itself validates their dimensions and labels but cannot document their exact numerical contents. This limits what can be concluded about model behavior from static inspection alone.

The second limitation concerns the Q1 target definition. The source explicitly preserves the original binary label file while constructing a separate three-class target from coordinate rules. This is a reasonable reconstruction of the assignment as documented in the code, but it creates two label representations for the same feature matrix. Future maintenance would benefit from making the derived target the canonical training label and treating the original binary vector explicitly as an archival consistency check.

The third limitation is the absence of a separate validation protocol. Both models use the first 70 samples for fitting and the final 30 for evaluation. This is correct for the stated assignment protocol but does not estimate variability across different splits. In Q2, the absence of regularization or early stopping also means the network relies on the fixed optimization schedule rather than a validation-driven training decision.

The fourth limitation is software packaging. The notebook installs packages dynamically when missing and uses hard-coded relative directories. This is convenient in Colab, but a research repository would be more robust with a pinned dependency specification, an explicit entry point, and separate modules for data, models, evaluation, and experiment configuration.

Finally, some imports and compatibility provisions are unused in the current source path. The unused `loadmat` import is notable because the prose mentions MATLAB-file loading during dataset preparation, whereas the executable data path consumes CSV files. This is not a correctness failure, but it adds minor conceptual noise that could be removed in a polished repository release.

13 Overall Technical Assessment

The supplied implementation is technically coherent as a small, assignment-focused reconstruction. Its main strength is that the corrective decisions are tied to the mathematical structure of the two problems rather than to superficial code changes. Q1 uses a genuine multiclass perceptron, while Q2 introduces an actual nonlinear hidden representation and manual backpropagation. The source also demonstrates awareness of data leakage, numerical stability, reproducibility, and artifact preservation.

From a software-engineering perspective, the code favors transparency over abstraction. That

is a defensible choice for a didactic notebook: the essential learning equations are visible, the control flow is easy to inspect, and the numerical operations map closely to the mathematical formulation. The trade-off is reduced modularity and limited reuse outside the assignment context.

From a research-methodology perspective, the implementation is deliberate about what it can control and reproduce: seed management, fixed splitting, training-only standardization, model-parameter persistence, and a manifest are all useful practices. At the same time, the absence of the datasets, environment pinning, and alternative evaluation splits means that the project should be presented as a reproducible educational experiment rather than as a statistically comprehensive machine-learning study.

The central technical idea connecting the two tasks is the distinction between the geometric capacity of a linear classifier and the representational value of a nonlinear hidden layer.

14 Conclusion

The project implements a complete two-part neural-classification workflow in Python and NumPy. It starts from validated two-dimensional datasets, reconstructs the intended three-class target for the first task, and trains a single-layer multiclass perceptron with a direct perceptron correction rule. It then standardizes the second dataset using training statistics only and trains a two-layer tanh/sigmoid network through explicit forward propagation, binary cross-entropy, backpropagation, and gradient descent.

The surrounding code adds practical infrastructure: deterministic initialization, input validation, explicit prediction and confusion-matrix routines, artifact serialization, a runtime manifest, and archive packaging. The implementation therefore shows not only familiarity with neural-network terminology, but also how the underlying algorithms can be expressed directly as numerical linear algebra and how the resulting experiment can be made inspectable and reproducible.

The report deliberately does not state or fabricate execution results because the supplied source contains result-generating code rather than fixed result values, and the external dataset files are not part of the submitted source. The technical conclusions are therefore limited to what the source itself supports.

A Complete Source-Coverage Traceability

The following map shows how the major line ranges of the 484-line source are represented in this report. It is included to make the analysis auditable without reproducing the source code.

Source lines	Role	Report treatment
1–19	File identity and assignment scope	Introduction and project overview
21–32	Audit findings and reconstruction decisions	Introduction, design decisions, and technical assessment
36–69	Package setup, imports, seed, directories	Technical foundations; implementation details; reproducibility
71–114	Dataset discovery, upload fallback, loading, validation	Data and input processing
114–133	Q1 consistency note and target reconstruction	Data and input processing; mathematical formulation
136–157	Split and Q2 standardization	Data and input processing; reproducibility
159–208	Q1 model, training, inference, accuracy, weights	Mathematical foundations; detailed methodology
210–220	Confusion-matrix utility and Q1 persistence	Evaluation and serialization
223–248	Q1 plotting utilities and saved figure	Auxiliary plotting path; omitted from report visuals by design
250–260	Q1 interpretation and Q2 motivation	Methodology and design decisions
263–314	Sigmoid, BCE, two-layer network, manual backpropagation	Mathematical foundations and detailed methodology

317–354	Q2 training, inference, evaluation, model/history persistence	Detailed methodology; execution workflow
356–400	Q2 history and boundary plotting	Auxiliary plotting path; omitted from report visuals by design
403–405	Q2 interpretation	Methodology and overall assessment
407–432	Runtime results summary	Reproducibility discussion; numerical results intentionally not reproduced
435–441	Final source conclusions	Overall technical assessment and conclusion
444–482	Manifest, archive, and Colab download	Serialization, execution workflow, and reproducibility
483–484	Non-Colab fallback output	Execution workflow

B Implementation Parameters Explicitly Present in the Source

Parameter	Value	Interpretation
Random seed	42	Controls model initialization and reproducibility pathways
Training/test split	70 / 30	First 70 samples train; final 30 samples test
Q1 number of classes	3	One score per geometric class
Q1 epoch limit	1000	Early termination occurs when an epoch has zero errors
Q1 initial weight/bias scale	0.1	Normal initialization scale for Q1
Q2 hidden units	12	Width of the nonlinear hidden layer
Q2 learning rate	0.03	Fixed gradient-descent step size
Q2 epochs	5000	Fixed full-batch training schedule
Q2 initial weight scale	0.5	Normal initialization scale for both trainable weight matrices
Q2 classification threshold	0.5	Converts sigmoid outputs to binary labels
Sigmoid input clip	$[-50, 50]$	Limits extreme exponential arguments
BCE probability clip	$[10^{-12}, 1-10^{-12}]$	Prevents $\log(0)$ in the loss